

Lógica Proposicional y Aplicaciones en Sistemas

¡Hola! Bienvenidos al fascinante mundo de la Lógica Proposicional.

Quizás te preguntes, ¿qué tiene que ver la lógica con programar, con las bases de datos o con las redes? La respuesta es: **todo**. La lógica es el esqueleto sobre el que se construye el software. Cada if, cada while, cada consulta a una base de datos (SELECT * FROM users WHERE isAdmin = TRUE) y cada regla de seguridad de un sistema se basa en los principios que vamos a estudiar.

Dominar la lógica te dará una herramienta fundamental para pensar de manera más clara, estructurada y para escribir código más eficiente y libre de errores.

1. ¿Qué es una Proposición?

El concepto central de nuestra unidad es la **proposición lógica**.

Una **proposición** es un enunciado declarativo (una afirmación) del cual se puede decir, sin ambigüedad, que es **Verdadero (V)** o **Falso (F)**.

A este valor (V o F) lo llamamos su **valor de verdad**. Es importante destacar dos principios fundamentales:

- **Principio de no contradicción:** Una proposición no puede ser verdadera y falsa al mismo tiempo.
- **Principio del tercero excluido:** Una proposición es o bien verdadera, o bien falsa; no hay una tercera opción.

¿Qué SÍ es una proposición?

- "La Universidad de Lanús fue fundada en el siglo XVIII." (Es una proposición. Su valor de verdad es **Falso**).
- "1024 es una potencia de 2." (Es una proposición. Su valor de verdad es **Verdadero**, ya que $2^{10} = 1024$).
- "Si un número es par, entonces es divisible por 2." (Es una proposición. Su valor de verdad es **Verdadero** por definición matemática).

¿Qué NO es una proposición?

1. **Preguntas:** No afirman nada, por lo tanto no pueden ser V o F.
 - "¿El servidor está respondiendo?"
2. **Órdenes o Súplicas:** Expresan un deseo o mandato, no un hecho.
 - "Por favor, compilá el código."
3. **Opiniones o Juicios de Valor:** Son subjetivos. Lo que es aburrido para uno puede ser fascinante para otro.
 - "Esta película es muy aburrida."
4. **Enunciados Abiertos:** Contienen variables. No son proposiciones hasta que la variable toma un valor específico.

- " $x + 3 = 7$." (Si $x=4$, la proposición " $4 + 3 = 7$ " es V. Si $x=1$, la proposición " $1 + 3 = 7$ " es F. Pero $x+3=7$ por sí solo no tiene valor de verdad).

2. Conectivos Lógicos: Construyendo Ideas Complejas

Rara vez trabajamos con una sola proposición. La verdadera potencia de la lógica aparece cuando combinamos proposiciones simples para formar **fórmulas complejas**. Para ello, usamos los **conectivos lógicos**. En programación, estas son análogas a los operadores $!$, $\&\&$ y $||$.

Usaremos letras minúsculas como p , q , r para representar proposiciones simples.

Conectiva	Símbolo	Lenguaje Natural	Analogía en Programación
Negación	$\neg$	"no", "no es cierto que..."	$!$
Conjunción	$\wedge$	"y", "además", "pero"	$\&\&$ (AND)
Disyunción	$\vee$	"o" (inclusiva)	$\vee$
Condicional	$\rightarrow$	"si... entonces...", "implica que..."	if (...) { ... }
Bicondicional	$\leftrightarrow$	"si y solo si"	$==$ (para booleanos)

a. Negación ($\neg$)

Simplemente invierte el valor de verdad de una proposición.

- p : "El jugador tiene la llave dorada." (V)
- $\neg p$: "El jugador **no** tiene la llave dorada." (F)

Tabla de Verdad:

p	$\neg p$
V	F
F	V

b. Conjunción ($\wedge$)

Es verdadera **únicamente** si ambas proposiciones que la componen son verdaderas.

- "El jugador tiene la llave dorada Y tiene más de 100 puntos de vida."
- **Simbólicamente:** $p \wedge q$

Tabla de Verdad:

p	q	$p \wedge q$
V	V	V
V	F	F
F	V	F
F	F	F

c. Disyunción ($\vee$)

Es verdadera si **al menos una** de las proposiciones es verdadera. Solo es falsa si ambas son falsas.

- "El jugador tiene más de 100 puntos de vida O tiene la llave dorada."
- **Simbólicamente:** $q \vee p$

Tabla de Verdad:

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

d. Condicional ($\rightarrow$)

Esta conectiva es crucial y a veces confusa. La fórmula $p \rightarrow q$ se lee "si p, entonces q".

- **p** se llama **antecedente** (la condición).
- **q** se llama **consecuente** (el resultado).

La implicación $p \rightarrow q$ es falsa **únicamente en un caso**: cuando el antecedente p es verdadero y el consecuente q es falso. Piensa en ello como una promesa: si yo prometo "Si aprobás el examen (p), te invito a comer (q)", ¿cuándo rompí mi promesa? Únicamente si vos aprobaste (p es V) y yo no te invité (q es F). En todos los demás casos, la promesa se mantiene.

- "Para estar en el nivel final, es necesario tener la llave dorada." Esto es una forma sutil de decir: Si estás en el nivel final, entonces tenés que tener la llave dorada.
- **Simbólicamente:** $p \rightarrow q$

Tabla de Verdad:

p	q	$p \rightarrow q$
V	V	V
V	F	F
F	V	V
F	F	V

e. Bicondicional ($\leftrightarrow$)

La fórmula $p \leftrightarrow q$ se lee "p si y solo si q". Exige que ambas proposiciones tengan el **mismo valor de verdad**. Es una implicación en ambos sentidos.

- "El jugador está en el nivel final si y solo si tiene la llave dorada y más de 100 puntos de vida."
- **Simbólicamente:** $r \leftrightarrow (p \wedge q)$

Tabla de Verdad:

p	q	$p \leftrightarrow q$
V	V	V
V	F	F
F	V	F
F	F	V

3. Tablas de Verdad: La Herramienta Definitiva

Para analizar fórmulas complejas, usamos **tablas de verdad**. Una tabla de verdad nos muestra el valor de verdad de una fórmula para **todas las combinaciones posibles** de los valores de verdad de las proposiciones simples que la componen.

¿Cuántas filas tiene una tabla? Si una fórmula tiene n proposiciones simples distintas, su tabla de verdad tendrá 2^n filas.

Pasos para construir una tabla:

1. Crear una columna para cada proposición simple (p, q, r, ...).
2. Crear columnas intermedias para las operaciones internas (ej: para $\neg q$, para $p \wedge q$, etc.), trabajando de adentro hacia afuera.
3. Crear la columna final para la fórmula completa.

Ejemplo: $(p \wedge \neg q) \vee p$

1. Columnas para p y q. ($2^2 = 4$ filas)
2. Columna para la operación más interna: $\neg q$.
3. Columna para la siguiente operación: $p \wedge \neg q$.
4. Columna final para toda la fórmula: $(p \wedge \neg q) \vee p$.

p	q	$\neg q$	$p \wedge \neg q$	$(p \wedge \neg q) \vee p$
V	V	F	F	V
V	F	V	V	V
F	V	F	F	F
F	F	V	F	F

4. Equivalencias Lógicas

Dos fórmulas son **lógicamente equivalentes** si sus tablas de verdad son idénticas en la columna final. Esto es muy útil para simplificar expresiones o para reescribir una condición de una manera que sea más fácil de entender o demostrar. Se denota con el símbolo $\equiv$.

Una de las equivalencias más importantes es la **contrapositiva** de un condicional:

$$p \rightarrow q \equiv \neg q \rightarrow \neg p$$

"Si llueve, entonces la calle se moja" es lógicamente equivalente a "Si la calle no está mojada, entonces no llovió". A veces, demostrar la contrapositiva es más sencillo que demostrar la implicación original.

Otra equivalencia fundamental es la que define al **bicondicional**:

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$$

Esto muestra que un "si y solo si" es realmente una doble implicación.

5. Lógica en Acción: De la Lógica al Código

Aquí es donde todo se conecta. Las condiciones lógicas que escribimos en un *if*, *while* o *case* son fórmulas proposicionales.

Ejemplo: Pseudocódigo y Lógica

Considera este código:

Fragmento de código

```
LEER esAdmin, tieneTokenValido, conexionEstable

SI (esAdmin Y tieneTokenValido) O (NOT conexionEstable) ENTONCES
    MOSTRAR "Acceso concedido"
SINO
    MOSTRAR "Acceso denegado"
FINSI
```

Podemos formalizar esta condición.

1. Definir proposiciones:

- p: "El usuario es administrador" (esAdmin)
- q: "El usuario tiene un token válido" (tieneTokenValido)
- r: "La conexión es estable" (conexionEstable)

2. Traducir la condición a lenguaje simbólico:

La condición (esAdmin Y tieneTokenValido) O (NOT conexionEstable) se traduce a:

$$(p \wedge q) \vee (\neg r)$$

Ahora, si sabemos los valores de verdad de p, q y r, podemos evaluar la fórmula para predecir el comportamiento del programa sin siquiera ejecutarlo. Esta es la base para el testing de software y la verificación formal de sistemas.

Guía de Actividades N° 1: Lógica Proposicional y Aplicaciones

Objetivos de la guía:

Al finalizar esta guía, el/la estudiante será capaz de:

- Identificar y diferenciar enunciados que son proposiciones de los que no lo son.
- Traducir enunciados del lenguaje natural al lenguaje simbólico de la lógica proposicional y viceversa.
- Construir y utilizar tablas de verdad para determinar el valor de verdad de fórmulas complejas.
- Evaluar y aplicar conectivos lógicos en el contexto de algoritmos simples expresados en pseudocódigo.
- Resolver problemas de razonamiento lógico de manera estructurada.

Actividad 1: Determiná cuáles de los siguientes enunciados son proposiciones lógicas. Para aquellos que lo sean, indicá si su valor de verdad es Verdadero (V), Falso (F) o si dependería de un contexto específico. Para los que no lo son, explicá brevemente por qué.

a) La Universidad de Lanús fue fundada en el siglo XVIII.

b) ¿El servidor está respondiendo?

c) Por favor, compilá el código.

d) 1024 es una potencia de 2.

e) $x + 3 = 7$.

f) Esta película es muy aburrida.

g) Si un número es par, entonces es divisible por 2.

Actividad 2: Considerá las siguientes proposiciones simples sobre el estado de un personaje en un videojuego:

- p: "El jugador tiene la llave dorada".
- q: "El jugador tiene más de 100 puntos de vida".
- r: "El jugador está en el nivel final".

Traducí las siguientes frases al lenguaje simbólico usando las proposiciones y conectivas lógicas:

a) El jugador no está en el nivel final.

b) El jugador tiene la llave dorada y tiene más de 100 puntos de vida.

c) Para estar en el nivel final, es necesario tener la llave dorada. (Pista: Pensá en la implicación).

d) Si el jugador tiene más de 100 puntos de vida o tiene la llave dorada, entonces no está en el nivel final.

e) El jugador está en el nivel final si y solo si tiene la llave dorada y más de 100 puntos de vida.

Actividad 3: Usando las mismas proposiciones (p,q,r) de la actividad anterior, traducí las siguientes fórmulas lógicas a un enunciado coherente en lenguaje natural:

a) q

$\rightarrow p$

b) $\neg p$

land

negq

c) *r*

leftrightarrow(p

landq)

Actividad 4: Construí las tablas de verdad para las siguientes fórmulas lógicas. Asegurate de incluir columnas intermedias para cada sub-operación.

a) *(p*

land

negq)

lorp

b) *p*

rightarrow(q

lor

negp)

c) *(p*

rightarrowq)

land(q

rightarrowp) (¿Reconocés a qué conectiva equivale esta fórmula?)

Actividad 5: Explorando Equivalencias Lógicas

a) Construí la tabla de verdad para la fórmula condicional:

p

rightarrowq.

b) Construí la tabla de verdad para su contrapositiva:

negq

rightarrow

negp.

c) Compará la columna final de ambas tablas. ¿Qué observás? Esta es una equivalencia lógica muy importante en las demostraciones.

Actividad 6: Dado el siguiente fragmento de pseudocódigo que determina si un usuario puede acceder a un panel de administrador:

```
Proceso ValidarAcceso
    Definir esAdmin, tieneTokenValido, conexionEstable Como Logico;

    // Asignación de valores para este caso de prueba
    esAdmin <- FALSO;
    tieneTokenValido <- VERDADERO;
    conexionEstable <- VERDADERO;

    SI ( (esAdmin O tieneTokenValido) Y conexionEstable ) Entonces
        Escribir "Acceso al panel concedido";
    Sino
        Escribir "Acceso denegado. No cumple los requisitos.";
    FinSi
FinProceso
```

a) Formalizá la condición del SI en lenguaje simbólico. Usá:

- p: "El usuario es administrador" (esAdmin).
- q: "El usuario tiene un token válido" (tieneTokenValido).
- r: "La conexión es estable" (conexionEstable).

b) Construí una tabla de verdad para la fórmula que escribiste en el punto (a).

c) Según los valores asignados en el código (p es F, q es V, r es V), ¿cuál será la salida del programa? Verificá tu respuesta usando la fila correspondiente de tu tabla de verdad.

Actividad 7: Una plataforma de e-commerce ofrece un descuento especial si se cumple alguna de las siguientes reglas:

1. El cliente es un usuario "Premium" **y** el total de la compra supera los \$10.000.
2. El cliente tiene un cupón de descuento válido.

a) Definí las proposiciones simples que necesitás.

b) Escribí la regla completa en una única fórmula lógica.

c) Escribí un fragmento de pseudocódigo que tome como entrada los valores lógicos de tus proposiciones y muestre por pantalla "Aplicar descuento especial" o "Sin descuento especial" según corresponda.

Actividad 8: Tres microservicios críticos de una aplicación (Servicio A: Autenticación, Servicio B: Base de Datos, Servicio C: Cache) dejaron de funcionar. El sistema de monitoreo alcanzó a registrar los siguientes mensajes de estado justo antes del colapso:

- **Servicio A reporta:** "El Servicio B falló".
- **Servicio B reporta:** "Yo no fallé".
- **Servicio C reporta:** "El reporte del Servicio A es falso".

Un análisis posterior reveló que **un solo servicio falló** y que, de los tres reportes, **solo el reporte del servicio que NO falló es verdadero**.

Tu tarea como DevOps es usar la lógica proposicional para descubrir:

1. ¿Cuál de los tres servicios (A, B o C) fue el que realmente falló?
2. ¿Cuál de los tres reportes es el verdadero?

Sugerencia: Definí proposiciones como p: "El Servicio A falló", q: "El Servicio B falló", etc. Luego, formalizá los reportes y analizá las tres hipótesis posibles (que A falló, que B falló, o que C falló) para ver cuál es consistente con las reglas del problema.